Lesson 4 SWD Simulation Debugging

1. Preface

After the program is written, it can be tested through simulation debugging. There are two debugging methods available on the STM32:

JTAG (Traditional Debugging Method): Primarily used for internal chip testing. Most advanced devices, such as DSPs and FPGAs, support the JTAG protocol.

SWD Debugging (Serial Debugging): A communication protocol for ARM core debuggers. Compared to the JTAG protocol, SWD debugging uses fewer port resources. This article will introduce the SWD debugging method.

2. Debugging Instructions

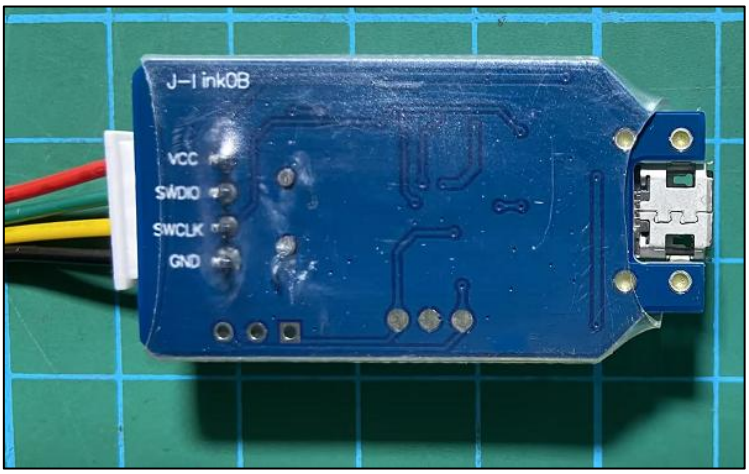
Software Preparation: Keil5 software (refer to previous sections for installation and compilation methods).

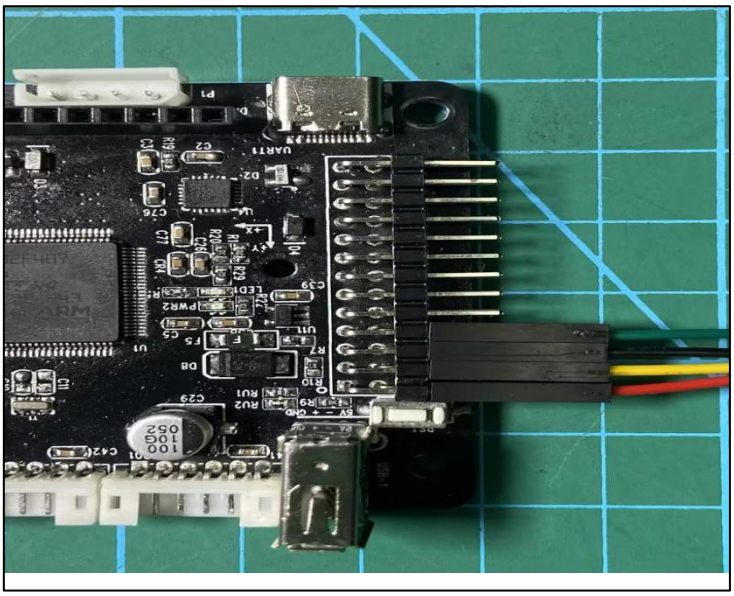
Jlink driver (found and installed from the "Appendix/Software Installation/Jlink Driver" folder).

Hardware preparation: Jlink emulator (self-supplied, with Dupont wires) and a MicroUSB cable.

2.1 Hardware Connection:

Connect Jlink to the stm32 controller as below:





The wiring guidelines are as follows:

VCC (red) -> 3V3

SWCLK (yellow) -> PA14

GND (black) -> GND

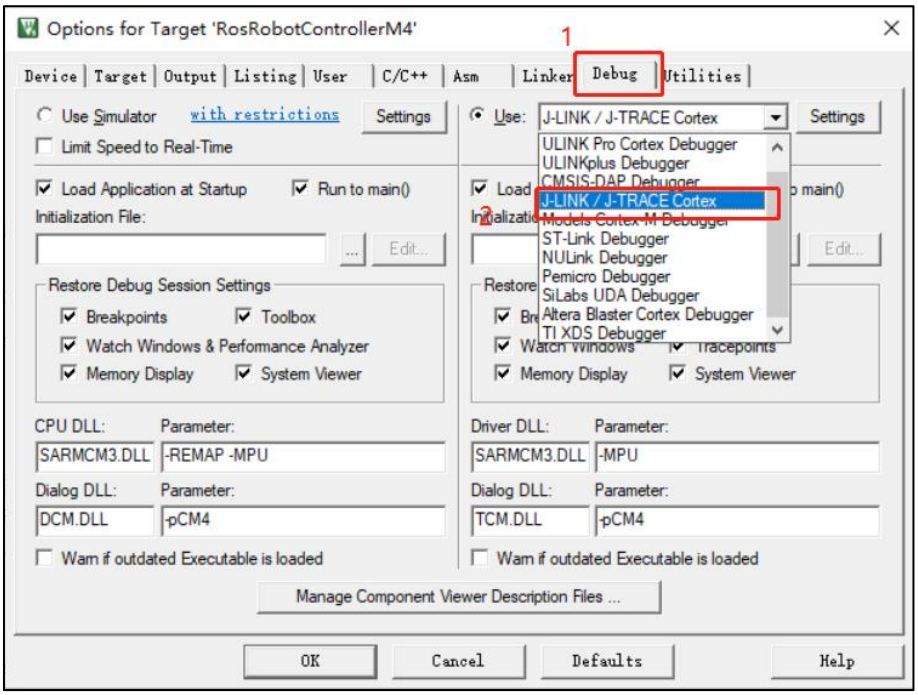
SWDIO (green) -> PA13

After completing the wiring, you can start software debugging.

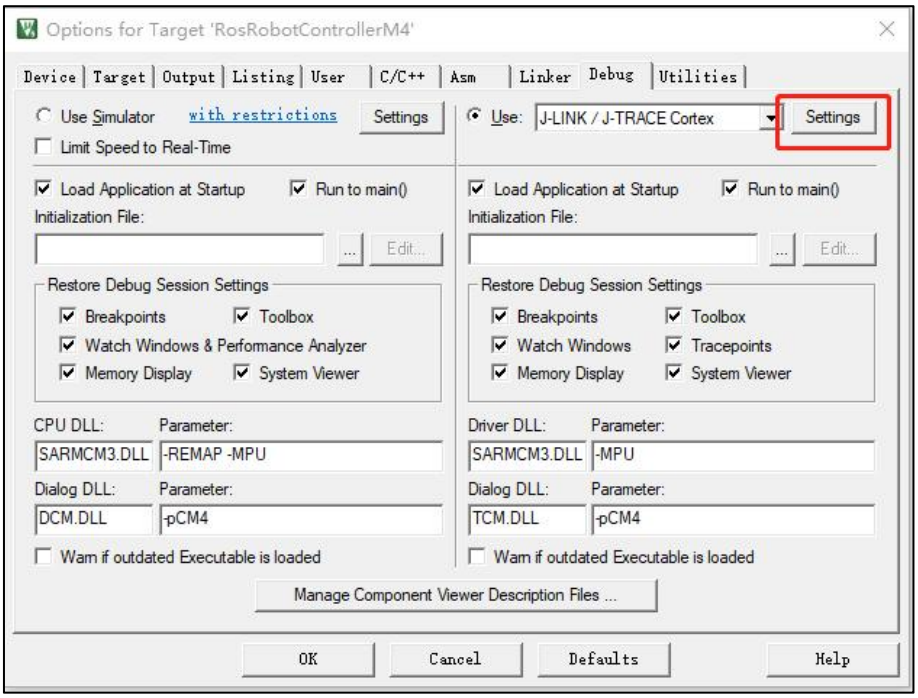
2.2 Set Up Debugging Environment

Open the Keil5 software and click the button on the icon to enter the environment configuration interface.

Click the "**Debug**" button, then select "**J-LINK/J-TRACE Cortex**" from the "**Use**" dropdown menu.

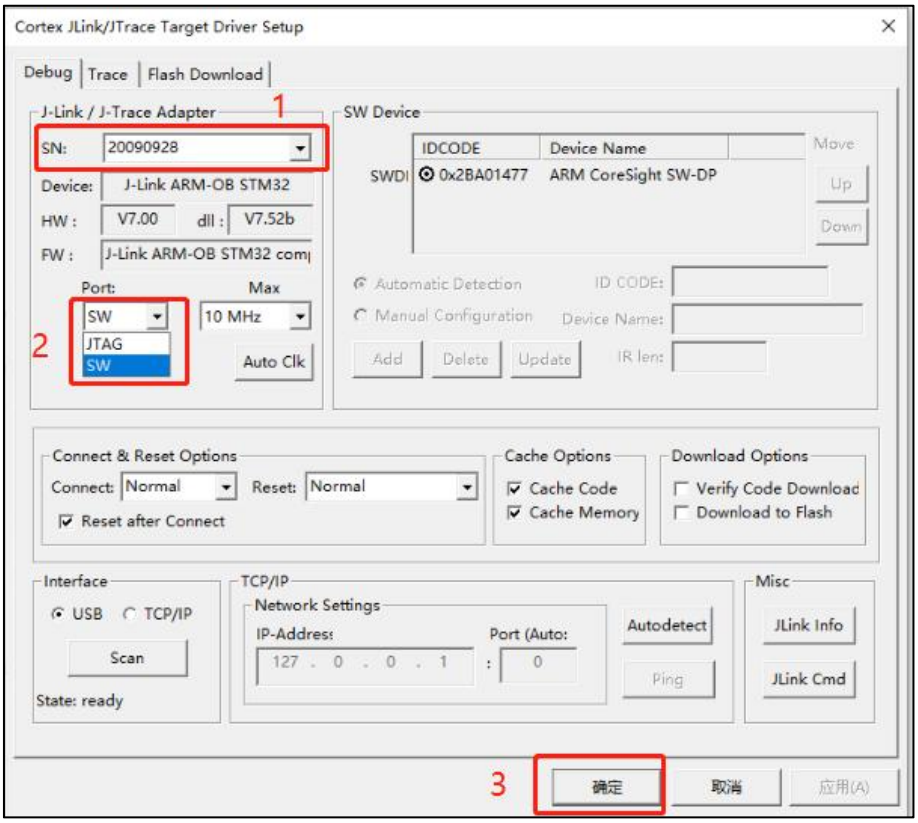


Click-on '**Settings**' to initiate configuration.

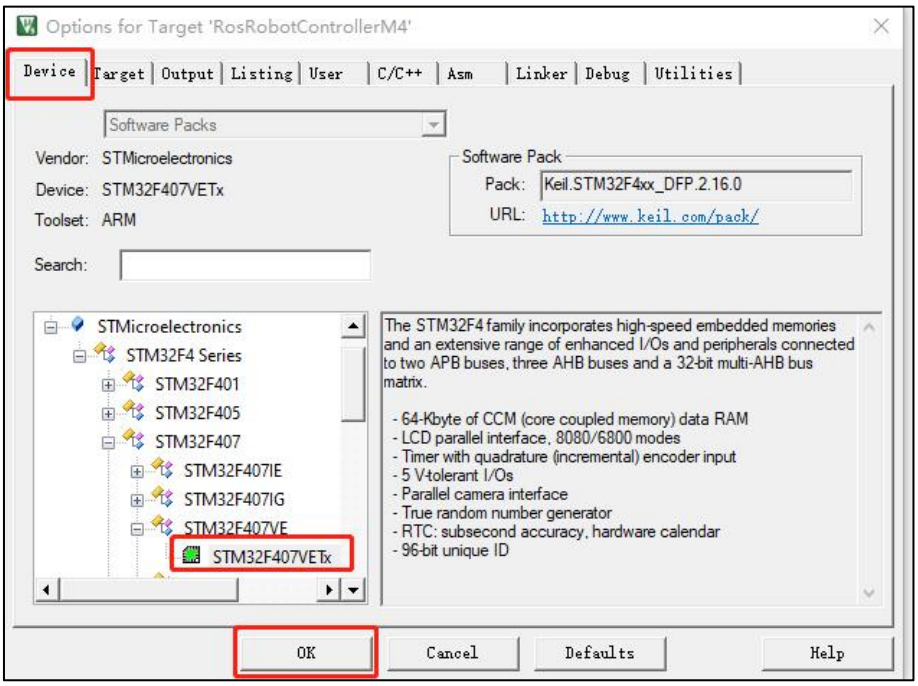


In the "**SN**" option, the serial number will be automatically recognized. Select it

or keep the default setting. Then, in the "**Port**" option below, select "**SW**" and finally click "**OK**".

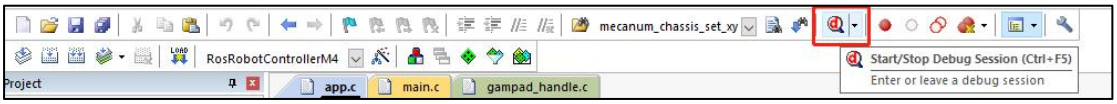


Return to the previous interface, click "Device", select the corresponding chip model, and then click "OK".

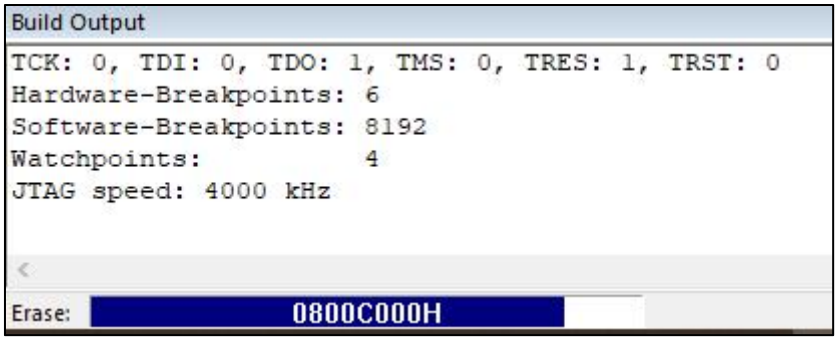


2.3 Debug

Click the "Debug" button on the Keil5 software interface to enter the debugging interface.

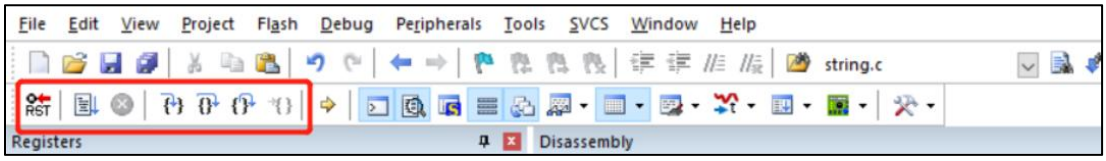


If the "Output" pane at the bottom displays the debugging loading process and no errors occur, it indicates normal operation.



>> Debugging Program Execution

In the debug interface toolbar (as shown below), the contents within the red box, from left to right:



Reset: Resets the program; the reset type triggered depends on the configuration of the burner.

Run: Starts the current program to run at full speed until it encounters a breakpoint.

Stop: When the program is running at full speed, clicking this button stops the program. It stops where the program is currently executing.

Step (F11): Executes a single statement according to the current debugging window's language. If encountering a function, it steps into the function. If in the disassembly window, it executes only one assembly instruction.

Step Over (F10): If in a C language window, it executes one statement at a

time. Unlike step debugging, it does not step into functions but runs them at full speed and jumps to the next statement.

Step Out: If in a C language window, it runs all the contents after the current function at full speed until returning to the previous level.

Run to Cursor Line.

Variable Checking

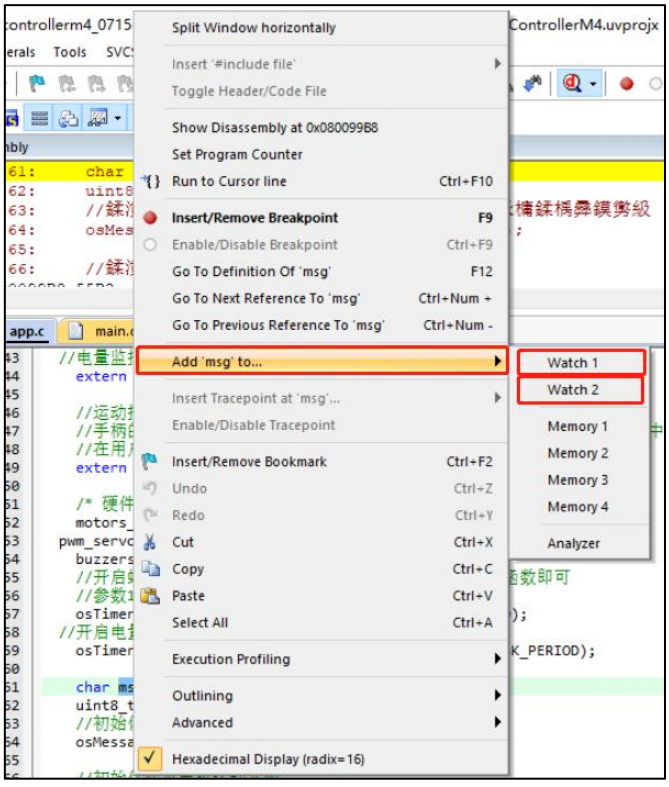
The button in the image above is the variable monitoring button, used to observe the assignment of variables during program execution.

Example:

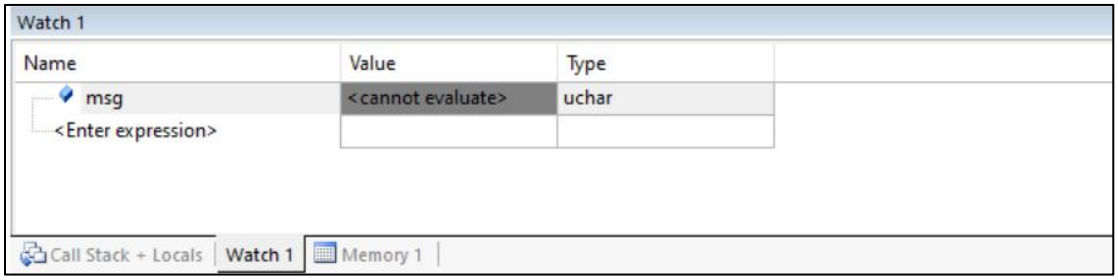
Pay attention to the msg variable in the program (users can choose the values they want to observe as examples during debugging).

```
char msg = '\0';
```

After selecting the msg variable, right-click and choose "Add 'msg' to", then select the observation window Watch1 or Watch2.



You can now observe the real-time value of msg in the Watch1 window below.



Note: To address any potential anomalies during the debugging process, users should follow the steps in order!